

Proyecto 2: Formulario de Contacto AJAX (XMLHttpRequest)

Objetivo del proyecto para los alumnos

- Aprender a enviar datos de un formulario **sin recargar la página** usando XMLHttpRequest.
- Procesar la respuesta del servidor y mostrarla dinámicamente en la página.
- Entender la **comunicación POST cliente-servidor**.
- Practicar **validaciones y mensajes dinámicos**.

1. Servidor Node.js con Express

1.1 Crear proyecto

```
mkdir proyecto_formulario
cd proyecto_formulario
npm init -y
npm install express cors
```

1.2 Servidor (**server.js**)

```
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors());
app.use(express.json()); // Para procesar JSON

// Ruta POST para recibir formulario
app.post('/api/contacto', (req, res) => {
  const { nombre, email, mensaje } = req.body;

  // Validaciones simples
  if (!nombre || !email || !mensaje) {
    return res.status(400).json({ success: false, msg: 'Todos los campos son obligatorios' });
  }

  // Simulación de procesamiento (p. ej. guardado en DB)
  console.log(`Mensaje recibido de ${nombre} (${email}): ${mensaje}`);

  // Respuesta al cliente
  res.json({ success: true, msg: `¡Gracias ${nombre}, tu mensaje ha sido recibido!` });
});
```

```
// Iniciar servidor
const PORT = 3001;
app.listen(PORT, () => {
  console.log(`Servidor escuchando en http://localhost:${PORT}`);
});
```

- **Puerto:** 3001
- **Ruta:** POST `http://localhost:3001/api/contacto`
- Devuelve un **JSON** con la respuesta.

Ejecutar servidor:

```
node server.js
```

2. Cliente HTML + Bootstrap

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Formulario de Contacto AJAX</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">

  <h1>Formulario de Contacto</h1>

  <form id="formContacto" class="mb-3">
    <div class="mb-2">
      <label for="nombre" class="form-label">Nombre</label>
      <input type="text" id="nombre" class="form-control"
name="nombre">
    </div>
    <div class="mb-2">
      <label for="email" class="form-label">Email</label>
      <input type="email" id="email" class="form-control"
name="email">
    </div>
    <div class="mb-2">
      <label for="mensaje" class="form-label">Mensaje</label>
      <textarea id="mensaje" class="form-control"
name="mensaje"></textarea>
    </div>
    <button type="submit" class="btn btn-success">Enviar</button>
  </form>

  <div id="respuesta"></div>

  <script src="script.js"></script>
</body>
</html>
```

3. JavaScript (`script.js`) usando XMLHttpRequest

```
const form = document.getElementById('formContacto');
const respuesta = document.getElementById('respuesta');

form.addEventListener('submit', function(e) {
    e.preventDefault(); // Evitar recarga

    const datos = {
        nombre: document.getElementById('nombre').value,
        email: document.getElementById('email').value,
        mensaje: document.getElementById('mensaje').value
    };

    // Crear objeto XMLHttpRequest
    const xhr = new XMLHttpRequest();
    xhr.open('POST', 'http://localhost:3001/api/contacto', true);
    xhr.setRequestHeader('Content-Type', 'application/json');

    // Función al recibir respuesta
    xhr.onload = function() {
        if(xhr.status === 200){
            const res = JSON.parse(xhr.responseText);
            if(res.success){
                respuesta.innerHTML = `<div class="alert alert-success">${res.msg}</div>`;
                form.reset();
            } else {
                respuesta.innerHTML = `<div class="alert alert-danger">${res.msg}</div>`;
            }
        } else {
            respuesta.innerHTML = `<div class="alert alert-danger">Error al enviar el formulario</div>`;
        }
    };

    // Manejo de errores
    xhr.onerror = function(){
        respuesta.innerHTML = `<div class="alert alert-danger">Error en la petición AJAX</div>`;
    };

    // Enviar datos como JSON
    xhr.send(JSON.stringify(datos));
});
```

4. Funcionamiento del proyecto

1. Ejecutar el servidor:

```
node server.js
```

2. Abrir `index.html` en el navegador.
3. Completar el formulario y hacer clic en **Enviar**.
4. La respuesta del servidor se muestra **dinámicamente** sin recargar la página.

5. Si falta algún campo, el servidor devuelve un error y se muestra en la página.

5. Ejercicios propuestos para los alumnos

1. Agregar validación en el **cliente**: verificar que el email tenga formato correcto antes de enviar.
2. Mostrar un mensaje “**Enviando...**” mientras llega la respuesta del servidor.
3. Guardar todos los mensajes enviados en un array en el servidor y crear una ruta `GET /api/contactos` para mostrarlos en una tabla.
4. Agregar un campo opcional “Teléfono” y mostrarlo en la respuesta si se envía.
5. Cambiar el diseño usando **Bootstrap cards** para mostrar el mensaje recibido.